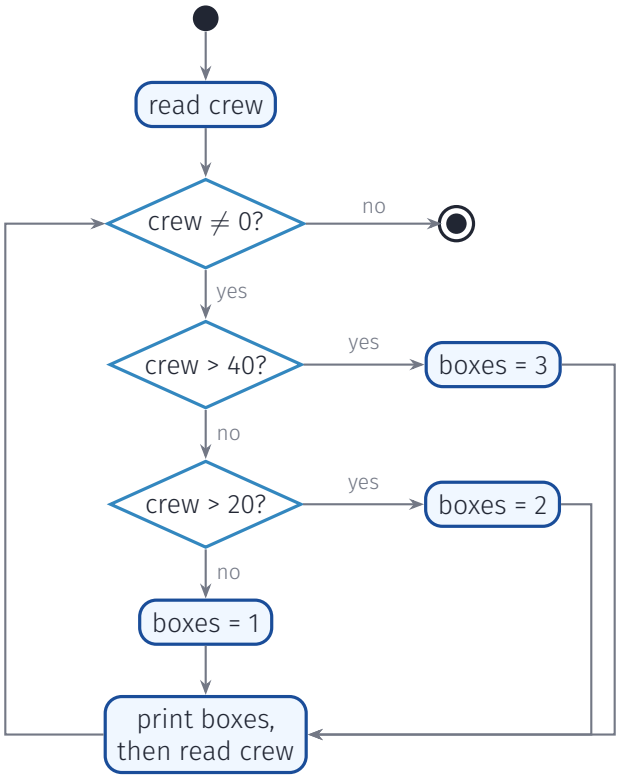
MIS41110 Practical Week 3, Solutions

Covers the Week 3 lecture (flow control, UML activity diagrams, pseudo-code) · **Worked** Monday of Week 4

Exercise 1: draw before you code



One variant worth having seen (it uses `continue`, a loop word that arrives in week 5, a preview, not an assignment): a `continue` inside the loop draws as **one more edge straight back to the decision diamond**, no new box. That is the entire notation for it.

(a) The diagram shows the **flow**: every path a run can take, and that the loop always returns to the same decision. The code shows the same thing one line at a time; the diagram shows it all at once.

Exercise 2: the tag validator

```
# Week-3 sheet, exercise 2 -- the tag validator.
# (a) while: we repeat UNTIL a condition holds; we cannot know how many tries.
tag = int(input("Tag number: "))
while tag < 1 or tag > 500:
    print("Tags run 1-500. Try again.")
    tag = int(input("Tag number: "))
print(f"Tag {tag} logged.")
```

(a) `while`: we repeat *until* a condition holds, and we cannot know in advance how many wrong answers will come. `for` is for a known number of steps.

Exercise 3: the comms menu

```
# Week-3 sheet, exercise 3 -- the comms menu loop.
choice = ""
while choice != "0":
    print("[1] Weather  [2] Colony count  [0] Sign off")
    choice = input("Choose: ")
    if choice == "1":
        print("Outside: -17.1 C, wind 35 km/h")
    elif choice == "2":
        print("North Ridge: 242 adelie penguins at last count.")
    elif choice == "0":
        print("Farpoint signing off.")
    else:
        print("Please choose 1, 2 or 0.")
```

Exercise 4: `for` and `range()` drills

```
# Week-3 sheet, exercise 4 -- for and range drills.
for number in range(10, 0, -1):
    print(number, end=" ")
print("LIFT")

for n in range(30):
    print("-", end="")
print()

vowels = 0
for letter in "Farpoint":
    if letter in "aeiou":
        vowels = vowels + 1
print(vowels)
```

Exercise 5: blizzard watch

```
# Week-3 sheet, exercise 5 -- blizzard watch.
# (a) The header has no "blizzard", so the count is safe today. It would break the
#     moment a header word matched what we count -- skip headers on principle.
notes = 0
for line in open("data/field_notes.txt"):
    if "blizzard" in line:
        notes = notes + 1
print(f"{notes} blizzard notes this season.")
```

(a) The header line has no `blizzard` in it, so today the count is safe. It would break the moment a header contained the word being searched for, which is why skipping the header deliberately is the habit, not counting on the data.

Exercise 6 ☆: one colony's file

```python
# Week-3 sheet, exercise 6 (starred) -- colony_extract.py.
import sys

if len(sys.argv) != 3:
    print("USAGE: colony_extract.py notes_file colony_name")
else:
    notes_file = sys.argv[1]
    colony = sys.argv[2]
    out_file = open(colony + ".txt", "w")
    written = 0
    for line in open(notes_file):
        if line.startswith(colony + ","):
            out_file.write(line)
            written = written + 1
    out_file.close()
    print(f"{written} notes written to {colony}.txt.")
```

Meaningful names, comments where needed, and test every branch: the happy path, the wrong path, and the boundary.